Name: Oluwatosin Ajayi

Student ID: 2413306

Department: Department of Computer Science

Module: Secure Software Development

Secure Healthcare Management Flask Application Design Using the SSDLC process.

Table of Contents

1.Introduction 4

2. Stage 1: Requirements Analysis & Initial Flask Setup 5

2.1 Stakeholder identification 5

2.2 Business objectives 6

2.3 Role-based requirements 6

2.4 Functional requirements 7

2.5 Security requirements 9

2.6 Ethical and professional requirements 11

2.7 Data sensitivity classification 11

3. Stage 2: Planning 12

3.1 Implementation Scope 12

3.2 Tools and Technologies 12

3.3 Risk Assessment 13

Stage 3: Design 16

4.1 Design Objectives 16

4.2 Architectural Overview 17

4.3 Modular Design of The System 17

4.4 Trust Boundaries 19

First Boundary – User Browser to Flask Application 19

Second Boundary - Flask Application to SQLite Authentication Database 20

Third Boundary - Flask Application to MongoDB Patient Records Database 20

Fourth Boundary - Administrative Functions 20

4.5 Data Design and Database Separation 21

4.6 Secure Design Decisions 21

4.7 STRIDE-Based Design View 23

Table of Tables

Table 1: Stakeholder Analysis 5
Table 2: Functional Requirements 7
Table 3: Security Requirements 9
Table 4: Data Sensitivity Classification 11
Table 5: Risk Assessment 13
Table 6: STRIDE Assessment Table 23

1.Introduction

Security is typically included in the last stages of traditional software development processes, making it the most difficult to deploy. This increases the risk of incurring additional expenses, lengthening development times, and harming a company's image (de Vicente Mohino et al., 2019).

Every stage of the software development lifecycle incorporates security measures. SSDLC guarantees that possible vulnerabilities are proactively addressed by integrating security from the very beginning of design all the way to deployment (Tal, 2020). There are nine SSDLC phases that largely resemble the SDLC model, with the exception that each step now takes security into account (Holdsworth & Badman, 2025):

- Requirements
- Analysis
- Planning
- Design
- Development
- Documentation
- Testing
- Deployment
- Maintenance

The aim of this project is to develop a secure web-based patient record management application for a private healthcare provider offering digital healthcare services. The proposed system will be designed according to the secure software development life cycle and will support three independent users: patients, clinicians and the administrator. After the designing the system, it should be able to allow authorised access to patient information, appointments, prescriptions, and healthcare records while protecting confidentiality, integrity, availability, and accountability.

2. Stage 1: Requirements Analysis

2.1 Stakeholder identification

In accordance with the defined scope of the project, a range of stakeholders has been identified. These stakeholders have been systematically categorized into primary and secondary groups based on their level of involvement and influence. A detailed list of the stakeholders is provided below:

Primary Stakeholders

- Patient
- Clinician
- Administrator

Secondary Stakeholders

- Healthcare Provider Organisation
- System Developer
- Regulator/ Legal Authority

The needs and security requirements of the primary stakeholders will be given more consideration because it aligns with the project scope. All roles, needs and security requirements of the stakeholders are represented in the table below:

Table 1: Stakeholder Analysis

Primary Stakeholders			
Stakeholder	**Role**	**Needs**	**Security Considerations**
Patient	Access personal Healthcare information	View booked appointments, prescriptions and personal health records	Privacy, confidentiality, identity protection
Clinician	Monitor and access patient records as required (i.e. diagnosis)	Manage medical records, create prescriptions and review healthcare history	High data accuracy, access authorisation, the system should be accountable
Administrator	Overall oversight on accounts and operations	User Management, role assignment and monitoring of system activity	Access control, auditable system, least privilege
Secondary Stakeholders			

Healthcare provider Organisation	Ownership and operation of the system	Adequate service delivery, statutory compliance, and oversight of operation	Legal compliance, Data governance, risk of reputational damage
System Developer	Design and implementation of the app	Secure design, maintenance and system documentation	Secure coding, accurate and adequate controls, testing
Legal Authority	Ensuring Compliance	Lawful and ethical handling of patient data	Protected data, high confidentiality and accountability

2.2 Business objectives

These are the business-related objectives that the proposed system must achieve. During the planning phase of this system, the following business objectives were identified:

a. Only authorised healthcare staff (namely clinicians) to securely manage and retrieve patient records to support diagnosis and ongoing monitoring
b. Patients should only access their personal health records
c. Provide adequate support for administrative workflows such as account management, oversight, and operational record handling.
d. Ensure sensitive healthcare data is protected against unauthorised access, misuse, accidental disclosure, and wrong modification
e. Traceable user actions and audit records for accountability particularly to patients.
f. Ensure legal, ethical and professional handling of healthcare data

2.3 Role-based requirements

For security purposes, it is important that user roles are defined and authorization is given based on defined roles within the app. These are the assigned roles used in the development of the secure web app:

All Users (Clinicians, Patients and Administrators)

- They are allowed to securely login in app

Patients

- All patients are allowed to view their own profile
- All patients are allowed to view their own appointments
- All patients are allowed to view own prescriptions
- All patients are allowed to view their medical summary

Clinicians

- Clinicians are allowed search patient records
- Clinicians are allowed to view patient full medical records
- Clinicians are allowed to manage consultation notes
- Clinicians are allowed to create prescription

Administrators

- Administrators are allowed to manage user accounts
- Administrators are allowed to assign roles
- Administrators are allowed to view audit logs

2.4 Functional requirements

The system's intended behaviour is captured by functional requirements. The services, activities, or functions that the system must carry out might be used to describe this behaviour (Malan et al., n.d.). The prospective healthcare management system must meet several functional requirements to be successfully operational. Relevant functional requirements are represented in the table below (IDs were used to foster traceability):

Table 2: Functional Requirements

ID	Functional Requirement	Priority
All Users		
FR1	The system will allow registered users to log in using username and password	Crucial
FR2	The system will identify the authenticated user's role and provide access only to role-appropriate functions	Crucial
FR3	The system shall record key user actions for accountability and audit purposes	Crucial
FR4	The system shall validate submitted	Crucial

	input before storing or processing it.	
FR5	The system shall provide logout functionality that ends the active user session securely.	Crucial
Patient		
FR6	The system shall allow patients to view only their own healthcare records and profile	Crucial
FR7	The system shall allow patients to view only their own appointments.	Crucial
FR8	The system shall allow patients to view only their own prescriptions	Crucial
Clinician		
FR9	Clinicians will be able to look up authorized patient records using the system.	Crucial
FR10	Clinicians will be able to examine patient medical records that are pertinent to treatment using the system.	Crucial
FR11	Clinicians will be able to write and modify consultation notes using the system.	Crucial
FR12	Clinicians will be able to write prescriptions for	Crucial

	patients using the system.	
Administrator		
FR13	Administrators will be able to create, modify, deactivate, or manage user accounts using the system.	Crucial
FR14	Administrators will be able to give each user account a single system role.	Crucial
System		
FR15	Data pertaining to authentication would be kept apart from patient medical information by the system.	Crucial

2.5 Security requirements

Software security standards are crucial criteria that guarantee programs are developed safely from the bottom up, shielding them from cyber-attacks and weaknesses. These specifications outline the security measures that must be incorporated throughout the development process to protect confidential information, stop illegal access, and adhere to industry rules (Miri, 2025). Recall the aim of this project is to develop a software while integrating security protocols in every design step. To effectively meet this aim, the following requirements will be essential:

Table 3: Security Requirements

ID	Security requirement	Why it is needed
SR1	Authentication is required before access to protected resources is granted.	It guarantees the prevention of unauthorized access to medical data.
SR2	Role-based authorisation must be enforced from the server side.	Stop administrators, clinicians, and patients from accessing each other's features
SR3	Only Patients should be allowed to access their own data.	Protect confidentiality and privacy

SR4	All user passwords are hashed before storage.	If the authentication database is exposed, safeguard the credentials.
SR5	User inputs are sanitised and validated before processing or storage	Minimize the chance of tampering and incorrect input
SR6	session controls are maintained to endure only authenticated sessions can access protected routes.	Stop unauthorized access and session misuse.
SR7	security-relevant actions such as login, record updates, and account management shall be logged by the system	Support accountability and auditability
SR8	The system must use suitable secure storage measures to safeguard sensitive medical data while it is being stored.	Minimize the chance of data leaking
SR9	The system would use several databases to keep patient clinical data and authentication data separate.	Minimize vulnerability and enhance security architecture
SR10	Only authorized administrator accounts will have access to privileged administrative operations.	Prevent privilege misuse
SR11	By restricting who may create, edit, or manage records, the system will protect data integrity.	Prevent unauthorised tampering
SR12	Through dependable management of key records and regulated system behaviour, the system will support availability.	Ensure clinical and operational use is not disrupted

2.6 Ethical and professional requirements

Given the diverse stakeholders involved in its operation and use, the proposed system's ethical and professional impact must be considered as a critical need. When defining the requirements the system must meet, the following appeared to be the most suited to satisfy ethical considerations:

Data Protection and Privacy

Healthcare data is extremely sensitive; therefore, the system should prioritize confidentiality, data protection, and privacy. To ensure that data is properly maintained, the system should restrict access to authorized users only and be built to minimize the needless exposure of patient information.

Traceability

Due to the nature of patient's records, there are always subject to change. However, the system must be designed to ensure that user actions affecting the patients recorded are traceable. This helps to ensure accountability.

Transparency

The system should be designed with transparency in view. Although security is imperative in the design process, all security processes should be documented to ensure transparency.

2.7 Data sensitivity classification

In today's digital world, when sensitive personal data gathering, processing, and storage are commonplace, data protection and privacy are essential ideas (Parry, 2025). Sensitivity levels should be allocated to all sensitive and personal data, and these levels should be considered during the design process to assist data protection and sensitivity.

Table 4: Data Sensitivity Classification

Data	Sensitivity Level	Justification
Username ID	Low	Identifies Account owner
Password	High	Loss leads to access to account
Patient full name	Medium	Personal details which identify owner
Patient ID	Medium	Connects owners' identity to medical record
Medical Details: Blood pressure, cholesterol, ECG, diagnosis-related data	High	Personal medical record

Medical history	High	Highly sensitive and personal records
Medical Prescriptions	High	Patients' prescriptions details are sensitive
Details of Appointment	Medium	Private health appointment information
Audit Logs	Medium	Contains details of access or change records
Assignment of User Roles	Medium	Wrongful role assignment can be detrimetnal

3. Stage 2: Planning

3.1 Implementation Scope

The project scope is based on the requirements set out in stage one (business, functional, security and ethical requirements). Following the completion of the project, the system should be able to support the patients, clinicians and administrators based on predefined roles and permissions.

Role-based access control will be used to apply these permissions. The patients will be allowed to access their personal healthcare records; the clinicians are allowed to update patients records as relevant to treatment and administrators are responsible for managing users accounts & managing the integrity of the system through account activation and deactivation. To reduce large-scale data exposure, the application to ensure that authentication data and sensitive health information by using multiple databases.

Lastly, the implementation will give priority to healthcare workflows such administrative control, patient record access, consultation management, and authentication. To maintain the system's focus, security, and alignment with assessment standards, non-essential elements are removed.

3.2 Tools and Technologies

To guarantee compliance with taught secure development methods, the system is constructed utilizing technologies described in the lesson. The backend is developed using Python and the Flask web framework. Flask is a backend framework written in Python. This approach allows you to design a functional application with minimal boilerplate code while giving attention to the unique characteristics of your project (Moore, 2025).

Two databases were used during the design of this application: SQLite and MongoDB. SQLite was used the primary database where all authentication data are stored. This database allows developers to store data without needing a separate database server (Hassan, 2025). MongoDB is used to store all healthcare-related data, including patient records, prescriptions, and consultation notes. Because MongoDB does not require a relational database management system (RDBMS), it offers an elastic data storage model that makes it easy for users to store and query multivariate data types (IBM, 2021). The use of multiple databases eliminates risks associated with storing data in a single location.

Finally, The Werkzeug library's hashing algorithms are used to achieve password security, guaranteeing that plaintext passwords are never saved. All user inputs are subjected to input validation procedures to guard against data insertion and corruption. Logging is used to assist accountability and monitoring by providing an audit record of important system occurrences.

3.3 Risk Assessment

Risk management in the Software Development Life Cycle (SDLC) is a methodical, proactive approach to detecting, addressing, and alleviating possible risks that impede the success of a software project. This entails early analysis of uncertainties and possible issues that may emerge at various phases of the SDLC process, together with the implementation of solutions for successful resolution and ongoing risk monitoring (Shergil, 2023).

Table 5: Risk Assessment

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R1	Unauthorised access to patient data	Medium	High	Establish role-based access control (RBAC) and impose server-side access verification for all routes.
R2	Weak authentication leading to account compromise	Medium	High	Implement password hashing (Werkzeug), enforce robust password

				validation, and utilize session-based authentication.
R3	Injection or malformed input attacks	Medium	High	Implement input validation on all user submissions via server-side validation mechanisms.
R4	Exposure of sensitive healthcare data due to poor data separation	Medium	High	Isolate authentication data (SQLite) from patient data (MongoDB)
R5	Privilege escalation (e.g., patient gaining admin access)	Low	High	Implement rigorous role verification decorators and authenticate roles at each secured endpoint.
R6	Lack of accountability for system actions	Medium	Medium	Establish logging for critical actions (login, logout, record modifications) while safeguarding sensitive information.
R7	Data tampering in patient records	Medium	High	Limit record alterations to authorized roles and verify record ownership.
R8	Denial of service through excessive requests	Low	Medium	Employ regulated request management and validation to minimize processing overhead.

R9	Hard-coded sensitive configuration (e.g., secret keys)	Medium	High	Transfer critical settings to environment variables or configuration files.
R10	Incomplete implementation of required features	Medium	High	Prioritize fundamental healthcare workflows at the first stages of development.
R11	Insufficient testing of security features	High	High	Implement unit and integration tests aligned with STRIDE threats
R12	Logging sensitive data (e.g., patient info or passwords)	Low	High	Ensure that logs exclusively comprise metadata, devoid of critical healthcare information.
R13	Ethical misuse of patient data	Low	High	Implement access control and adhere to data minimization standards.

Risks indicate a correlation between system performance and security vulnerabilities. Unauthorized access, data leakage, and privilege escalation pose significant challenges given the sensitive nature of healthcare data. The security of the healthcare system is contingent upon secrecy and integrity, which are jeopardized by these threats.

Mitigation methods were selected utilizing module lecture approaches. Position-based access control restricts users to accessing data pertinent to their role, hence minimizing information leakage. Password hashing prevents the storing of unencrypted credentials, hence reducing the danger of database breaches.

Stage 3: Design

4.1 Design Objectives

The design phase of the Secure Software Development Life Cycle (SSDLC) involves creating a secure architecture that incorporates defense-in-depth principles and minimizes attack surfaces (Reddy, 2025). At this step, the previously specified business, functional, and security requirements are converted into a secure technological framework for the healthcare application. The design phase of the Secure Software Development Life Cycle (SSDLC) entails establishing a secure architecture that integrates defense-in-depth concepts and reduces attack surfaces.

The design of the suggested healthcare system is informed by the subsequent objectives:

- to safeguard the confidentiality of patient information
- to uphold the integrity of healthcare records, prescriptions, and account information
- to ensure system availability for authorized clinical and administrative purposes
- to implement role-based access controls for patients, clinicians, and administrators
- to guarantee accountability via audit logging
- to mitigate the consequences of a breach by isolating authentication data from healthcare information
- to facilitate maintainability and testing through modular architecture

These objectives directly facilitate the business aim of enabling physicians to maintain treatment records, permitting patients to access just their personal information, and allowing administrators to securely oversee account management and oversight activities.

4.2 Architectural Overview

A definitive architectural framework is essential for successful security emphasis (Orhan Caylak, 2026). The proposed system has a three-tier web application architecture comprising:

1. Presentation
   This layer delivers the user interface using Flask templates and HTML documents. It oversees user input, form submission, and the presentation of records, appointments, prescriptions, dashboards, and messages.

2. Application
   This layer encompasses the logic of the Flask application. It is accountable for:
   - Authentication and session management
   - Authorization and role verification
   - Input validation
   - Business regulations
   - Routing
   - Audit logging
   - Database communication

3. Database
   This layer retains system data in two distinct databases:
   - SQLite for authentication and account-related information
   - MongoDB for managing patient healthcare records, medicines, appointments, and consultation notes.

4.3 Modular Design of The System

"Modularity in design" divides a system into distinct modules or skids that may be created independently and employed in other systems. Modular systems utilize functional partitioning into distinct, scalable, reusable modules, well specified modular interfaces, and established industry interface standards (Salimi & Salimi, 2018). The system is segmented into smaller sections to facilitate modular design and minimize coupling. The primary application components are:

1. Authentication
   Responsible for:
   - user login

- password verification
- session creation
- logout
- route protection for authenticated users

2. Authorisation
   Responsible for:
   - role checks
   - access control enforcement
   - blocking unauthorised access to dashboards and protected functions
   - ensuring patients only access their own data

3. Patient
   Responsible for:
   - viewing patient profile
   - viewing own appointments
   - viewing own prescriptions
   - viewing own medical summary
4. Clinician
   Responsible for:
   - creating and managing user accounts
   - assigning exactly one role to each user
   - activating or deactivating accounts
   - viewing audit-related information
5. Administrator
   Responsible for:
   - creating and managing user accounts
   - assigning exactly one role to each user
   - activating or deactivating accounts
   - viewing audit-related information
6. Validation
   Responsible for:
   - checking data type
   - checking format and length
   - whitelist-style validation where appropriate
   - rejecting malformed or unsafe input before processing
7. Logging
   Responsible for:
   - recording authentication events
   - logging access denials

- logging account management actions
- logging record create/update/delete actions where appropriate
- avoiding the logging of sensitive content such as plaintext passwords

4.4 Trust Boundaries

A trust boundary defines a demarcation where differing levels of trust exist. The network may provide a trust border, as access to the Internet is universally available, although not all individuals should have access to the corporate system (Myagmar et al., 2005). Secure architectural design necessitates the unambiguous delineation of boundaries to facilitate the implementation of appropriate controls at each transition.

First Boundary – User Browser to Flask Application

This constitutes the primary external trust border. Users input credentials, search queries, patient identification, consultation notes, and more form data via the browser into the Flask application.

Risks associated with this boundary

- forged login attempts
- malicious or corrupted input
- unauthorized requests
- session exploitation

Controls positioned here

- authenticity verifications
- server-side input validation
- session-oriented access control
- route safeguarding
- role authentication
- login failure management

Second Boundary - Flask Application to SQLite Authentication Database

This boundary manages account and credential-related information.

Risks associated with this boundary:

- credential breach
- role abuse
- account manipulation

Controls positioned here:

- hashing passwords prior to storage
- regulating database access via application logic
- exclusive account management for administrators
- recording of security-critical account activities

Third Boundary - Flask Application to MongoDB Patient Records Database

This border manages the most sensitive healthcare information. Given the clinical sensitivity of patient information, medications, and consultation notes, access to this database must be rigorously limited at the application layer.

Risks associated with this boundary

- unauthorized record access
- record alteration
- disclosure of medical data
- excessive data exposure

Controls positioned here

- server-side authorization
- patient ownership verification
- clinician-exclusive record modification capability
- data segregation from authentication information
- audit logging for record-related activities

Fourth Boundary - Administrative Functions

Administrative functions define a privilege boundary, as administrators possess the authority to create users, assign roles, and manage account status.

Risks associated with this boundary

- escalation of privileges
- abuse of administrative functions
- unauthorized role allocation

Controls positioned here:

- administrative-exclusive route decorators
- documentation of administrative activities
- distinct delineation of administrative pathways from those of patients and clinicians

4.5 Data Design and Database Separation

The system uses two databases to achieve data segregation. Numerous considerations warrant data segregation. Primarily, it protects essential data. Data can be partitioned to limit access to authorized individuals. This prevents data breaches and unauthorized access. Secondly, data segmentation enhances efficiency. Data is more manageable and organized when compartmentalized. This facilitates data discovery and enhances system performance (*PrivacyEngine*, 2023).

SQLite was used to store user accounts, usernames, hashed passwords, assigned roles, account status. Due to the structured and relational nature of the data, SQLite is appropriate here.

Healthcare information was stored using MongoDB due to the possibly of the information changing overtime. This database allows these information to be stored in flexible structures.

4.6 Secure Design Decisions

Decisions in Secure Design

The subsequent design choices were derived directly from the requirements of the healthcare context and the security concepts presented in the module.

1. Authentication Prior to Access

All protected resources need successful authentication prior to access being provided. This serves as the first line of defense in the system, preventing unauthorized access to patient or administrative data. This design accommodates SR1.

2. Server-Side Role-Based Authorization

The solution implements authorization on the server side instead of depending just on interface visibility. Patients, clinicians, and administrators possess distinct roles and

responsibilities. This design accommodates SR2 and SR10.

3. Verification of Ownership for Patient Data
A patient should only access their own profile, medicines, appointments, and health summary. This constitutes a more robust control than mere role verification, as it inhibits one patient from accessing another patient's information. This accommodates SR3 and FR6–FR8.

4. Password Hashing
Passwords are kept as hashed values instead than plaintext. This safeguards credentials in the event of authentication database exposure and demonstrates secure credential management as discussed in the lectures. This is compatible with SR4.

5. Input Validation
All user input must undergo validation on the server side prior to processing or storage. Validation encompasses data type, format, allowable characters, and length. This is compatible with SR5 and FR4.

6. Management of Sessions
The system employs authenticated sessions to preserve user state post-login and to limit access to secured routes. Session termination upon logout is essential to avoid unauthorized access continuing. This is compatible with SR6 and FR5.

7. Documentation and Accountability

The system records significant security-related events, including login attempts, role-specific activities, and record alterations. Logs enhance accountability and facilitate the investigation of misuse; nevertheless, they must exclude sensitive information and superfluous personal data. This complies with SR7 and the ethical mandate for traceability.

8. Secure Data Segregation
Authentication information and medical records are maintained independently. This restricts vulnerability in the event of a database breach and facilitates more precise access management. This is compatible with SR8 and SR9.

9. Minimal Privilege for Administrative Functions
Administrative acts are classified as privileged operations and are accessible solely via administrator-exclusive pathways. This diminishes the likelihood of privilege abuse and conforms to the principle of least privilege. This is compatible with SR10.

4.7 STRIDE-Based Design View

STRIDE is an acronym representing Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. This framework and mnemonic were created to assist software developers in recognizing the many forms of threats that software commonly encounters (Shostack, 2014). To successfully design a secure system, these threats were analysed and mitigations for each threat was worked into the system. The table below contains the final result of the analysis:

Table 6: STRIDE Assessment Table

Asset / Process	STRIDE Category	Example Threat	Security Impact	Design Mitigation
Login process	Spoofing	Attacker attempts to sign in as a valid user	Unauthorised access	Authentication, password hashing, session control
Patient record update	Tampering	Unauthorised modification of consultation notes or prescriptions	Loss of integrity, clinical risk	Role checks, ownership checks, input validation, restricted update routes
Audit history	Repudiation	User denies performing an action	Weak accountability	Security logging with timestamps and event context
Patient record retrieval	Information Disclosure	Patient or attacker views another patient's data	Loss of confidentiality	Role-based access control, ownership enforcement, data segregation
Login or search requests	Denial of Service	Malformed or repeated requests reduce service availability	Reduced availability	Input validation, controlled request processing, simpler route design
Admin role management	Elevation of Privilege	Non-admin gains admin capability	Loss of control over the system	Admin-only routes, server-side role validation, logging

Stage 4: Testing

4.1 Testing Strategy

The system was tested after it was designed to make sure the ethical, security, and functional criteria were correctly applied. This stage of the Secure Software Development Life Cycle is essential. The testing strategy used is centered on verifying:

- The efficiency of input validation procedures,
- The accuracy of fundamental system functions,
- The enforcement of authorization and authentication procedures
- The session management's resilience
- The system's capacity to mitigate risks found using the STRIDE framework

4.2 Functionality Testing

The system must meet all functional requirements stated in the requirements section. To verify this, the system was tested to ensure that all the functional requirements (FR1-FR15) were met across different user roles. The following areas were tested:

- User authentication, encompassing login and logout procedures.
- Role-specific access to system dashboards
- Individualized access to personal medical records
- Clinician's capacity to search, examine, and modify patient records
- Administrator's functionalities for user account administration and role allocation.

4.3 Security Testing

Security requirements were established during the requirements phase to guarantee the system's robustness. Furthermore, threats were detected through the risk and STRIDE study, and the findings from these analyses were utilized in the system's design. Security testing was conducted to assess the efficacy of the installed security measures.

The primary focus of the security testing process includes:

- implementing authentication prior to granting access to secured resources
- Validating server-side role-based access control,
- ensuring users access only authorized data,
- checking input validation and rejecting incorrect data,
- confirming secure session management,

- assuring the documentation of security-relevant activities for accountability

4.3.1 STRIDE Test

The table below outlines the correlation between STRIDE threat categories, applied security measures, and the associated test cases utilized for system validation.

Table 7: STRIDE Test Table

STRIDE Category	Threat Scenario	Implemented Control	Test Case Description	Expected Result
Spoofing	Attempt to log in using invalid credentials	Authentication and password hashing	Submit incorrect username and password	Access is denied and login fails
Spoofing	Unauthenticated user attempts to access protected route	Session validation	Access dashboard without login	User is redirected to login page
Tampering	Submission of invalid or malformed input	Input validation and format checks	Submit incorrect data format or unexpected input	Input is rejected and not processed
Tampering	Attempt to modify another patient's record	Ownership verification and RBAC	Access patient record not assigned to user	Access is denied
Repudiation	User denies performing an action	Logging of system activities	Perform login or record update and check logs	Action is recorded with timestamp
Information Disclosure	Patient attempts to access another patient's data	Role-based access and ownership enforcement	Attempt access using different patient identifier	Access is denied
Information Disclosure	Non-admin attempts to access administrative features	Server-side role validation	Access admin route as non-admin user	Access is denied
Denial of Service	Repeated invalid or malformed requests	Input validation and controlled processing	Submit repeated invalid inputs	System handles requests without failure

Elevation of Privilege	Non-admin user attempts admin actions	Role-based access control	Attempt to access admin functionality	Access is denied
Elevation of Privilege	Attempt to manipulate user role assignment	Restricted admin-only operations	Modify role without admin privileges	Modify role without admin privileges

5.Conclusion

The built healthcare management system exemplifies robust adherence to secure software development principles, successfully integrating business requirements, threat modelling, and deployed security measures to safeguard sensitive patient information. The system effectively implements authentication, role-based access control, input validation, session management, and audit logging, therefore reducing significant risks such as spoofing, tampering, and information leakage. Although certain residual concerns persist, such as inadequate defense against denial-of-service attacks and possible insider exploitation, the design demonstrates suitable compromises among security, usability, and system complexity. Thus, the application offers a safe and effective solution that fulfils the goals of the SSDLC and lays a robust groundwork for future improvements and practical implementation.

6.References

de Vicente Mohino, J., Bermejo Higuera, J., Bermejo Higuera, J. R., & Sicilia Montalvo, J. A. (2019). The Application of a New Secure Software Development Life Cycle (S-SDLC) with Agile Methodologies. *Electronics*, *8*(11), 1218. https://doi.org/10.3390/electronics8111218

Hassan, A. (2025, April 24). *What Is SQLite? (Definition, Use, Example, Alternatives) | Built In*. Builtin.com. https://builtin.com/data-science/sqlite

Holdsworth, J., & Badman, A. (2025, January 19). *The secure software development lifecycle*. Ibm.com. https://www.ibm.com/think/topics/secure-software-development-life-cycle

IBM. (2021, October 14). *What Is MongoDB?* Ibm.com. https://www.ibm.com/think/topics/mongodb

Malan, R., Company, H.-P., & Bredemeyer, D. (n.d.). *Functional Requirements and Use Cases Functional Requirements*. Retrieved March 19, 2026, from http://www.uml.org.cn/RequirementProject/pdf/functreq.pdf

Miri, M. (2025, March 9). *What Are Software Security Requirements?* Security Compass. https://www.securitycompass.com/blog/what-are-software-security-requirements/

Moore, V. (2025, July 6). *What Is Flask? A Comprehensive Guide to the Python Backend Framework*. MetaCTO; Meta CTO. https://www.metacto.com/blogs/what-is-flask-a-comprehensive-guide-to-the-python-backend-framework

Myagmar, S., Lee, A., & Yurcik, W. (2005). *Threat Modeling as a Basis for Security Requirements*.

Orhan Caylak. (2026). *Secure by Design: Principles for a Secure Architecture*. Arvato-Systems.com; Arvato Systems. https://www.arvato-systems.com/blog/secure-by-design-principles-for-a-secure-architecture

Parry, G. (2025, December 5). *Data Protection and Privacy in Software Development*. ResearchGate. https://doi.org/10.13140/RG.2.2.11804.14728

PrivacyEngine. (2023). PrivacyEngine Data Protection Software and Consultancy. https://www.privacyengine.io/resources/glossary/data-segregation/

Reddy, A. (2025). *Secure Software Development Life Cycle (SDLC): Integrating Security into Every Phase*. https://admin.mantechpublications.com/index.php/JoSEST/article/view/2080/633

Salimi, F., & Salimi, F. (2018). System Engineering of the Complex Megaprojects. *A Systems Approach to Managing the Complexities of Process Industries*, 221–277. https://doi.org/10.1016/b978-0-12-804213-7.00004-9

Shergil, A. (2023, July 20). *How to Manage Risk In The Software Development Life cycle?* TopDevelopers.co. https://www.topdevelopers.co/blog/risk-management-in-software-development/#what-is-risk-management-in-sdlc

Shostack, A. (2014). STRIDE. In *Threat Modeling: Designing for Security* (pp. 61–86). Wiley. http://ieeexplore.ieee.org/document/9932336

Tal, L. (2020, June 21). *Secure Software Development Lifecycle (SSDLC)*. Snyk. https://snyk.io/articles/secure-sdlc/

7.Appendix